

Face Class Reference

The Face class represents a face of a 3D object. [More...](#)

```
#include <face.h>
```

Public Types

```
enum infoInSource { VONLY, VN, VT, VTN }
```

Public Member Functions

	Face ()
	Face (int i0, int i1, int i2, int i3=-1)
void	addVertexIndex (int i)
int	numVertices () const
int	vertexIndex (int i) const
Vector	normal () const
void	computeNormal (const vector< Vertex > &verts)
infoInSource	faceInfo () const
int	normalIndex (int i) const
int	texcoordsIndex (int i) const

Friends

```
class Object
```

Detailed Description

Each face keeps a list of vertices (actually vertex indices) and a normal vector. The face is assumed to be convex.

Member Enumeration Documentation

enum Face::infoInSource

Enumerated type to describe the information for the face originating from the file.

Enumerator	
VONLY	Model file contained only vertex coordinates
VN	Model file provided coordinates and normals
VT	Model file provided coordinates and texture coordinates
VTN	Model file provided coordinates, normals and texture coordinates

Constructor & Destructor Documentation

Face::Face ()

Constructs an empty face.

```
Face::Face ( int i0,  
             int i1,  
             int i2,  
             int i3 = -1  
            )
```

Constructs a face with 3 or 4 vertices (actually vertex indices).

Member Function Documentation

void Face::addVertexIndex (int i)

Adds a new vertex index i to the face (at the end, i.e. push_back).

```
void Face::computeNormal ( const vector< Vertex > & verts )
```

Recomputes the normal vector of the face. This method must be called explicitly after constructing/modifying the face.

Parameters

verts contains actual vertex coordinates from the object containing the face.

```
Face::infoInSource Face::faceInfo ( ) const
```

returns a constant enum indicating the info that was contained originally in the file. Other info may be available nonetheless, but will have been computed by the driver

```
Vector Face::normal ( ) const
```

Returns the current normal vector of the face (undefined by default). The return value is thus undefined unless **computeNormal()** has been called for this face.

```
int Face::normalIndex ( int i ) const
```

returns the position of the vertex normal in the object's vector of normals. (a reference to that vector is returned by **Object::vertNormals()**)

Parameters

i index of the vertex for which we want to retrieve the normal; must be in 0..2

```
int Face::numVertices ( ) const
```

Returns the number of vertices forming the face.

```
int Face::texcoordsIndex ( int i ) const
```

returns the position of the texture coordinates for a vertex in the object's vector of same. (a reference to this vector may be obtained with **Object::vertTexCoords()**)

Parameters

i index of the vertex for which we want to retrieve the texture coordinates; must be in 0..2

```
int Face::vertexIndex ( int i ) const
```

Returns the vertex index at the i-th position. First vertex has index 0.